

宁波大学
NINGBO UNIVERSITY

实 验 报 告

实验五《最短路径算法》

学生姓名 顾凯琪

学号 226001624

课程名称 数据结构与算法

实验学期 2023 至 2024 学年 第 二 学期

所在学院 信息科学与工程学院

专业班级 计算机科学与技术 2 班

任课教师 赵杰煜

实验成绩

一、实验题目

最短路径算法

二、实验目的：

以全国主要城市为图的顶点，铁路连接为图的边，距离作为加权，设计完成一个最短路径自动查找系统. 输入为出发城市和目标城市，输出为最短路径和距离。

三、实验设备与环境

微型计算机、Windows 系列操作系统 、Visual Studio、Pycharm 系列软件

四、实验内容

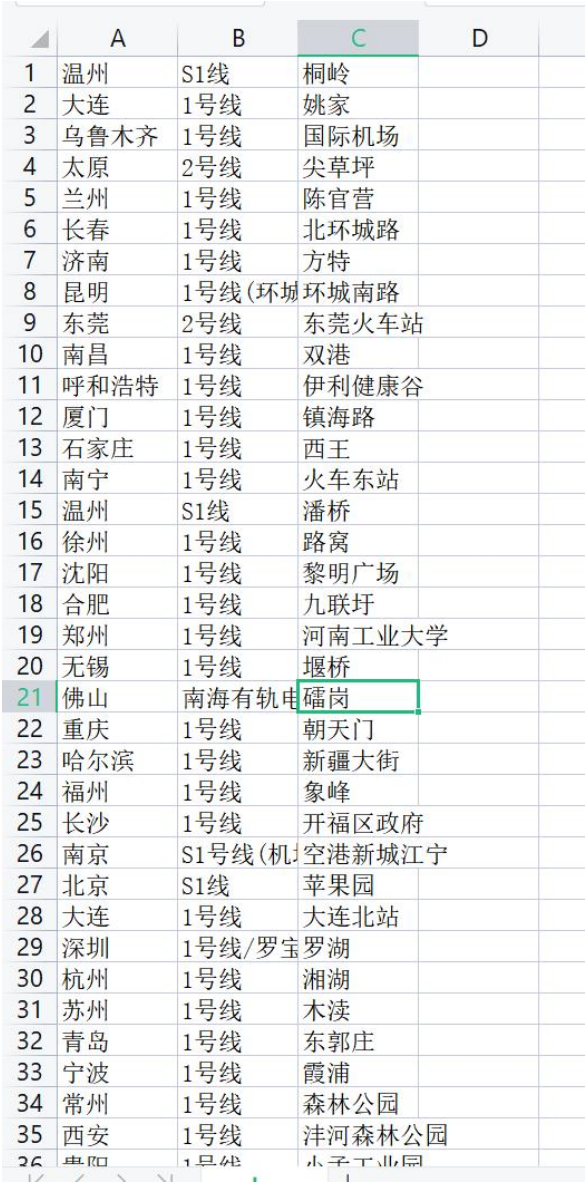
以全国主要城市为图的顶点，铁路连接为图的边，距离作为加权，设计完成一个最短路径自动查找系统. 输入为出发城市和目标城市，输出为最短路径和距离.

五、概要设计（思路、算法、步骤等）

首先使用百度地图的 api，可以成功调用百度地图

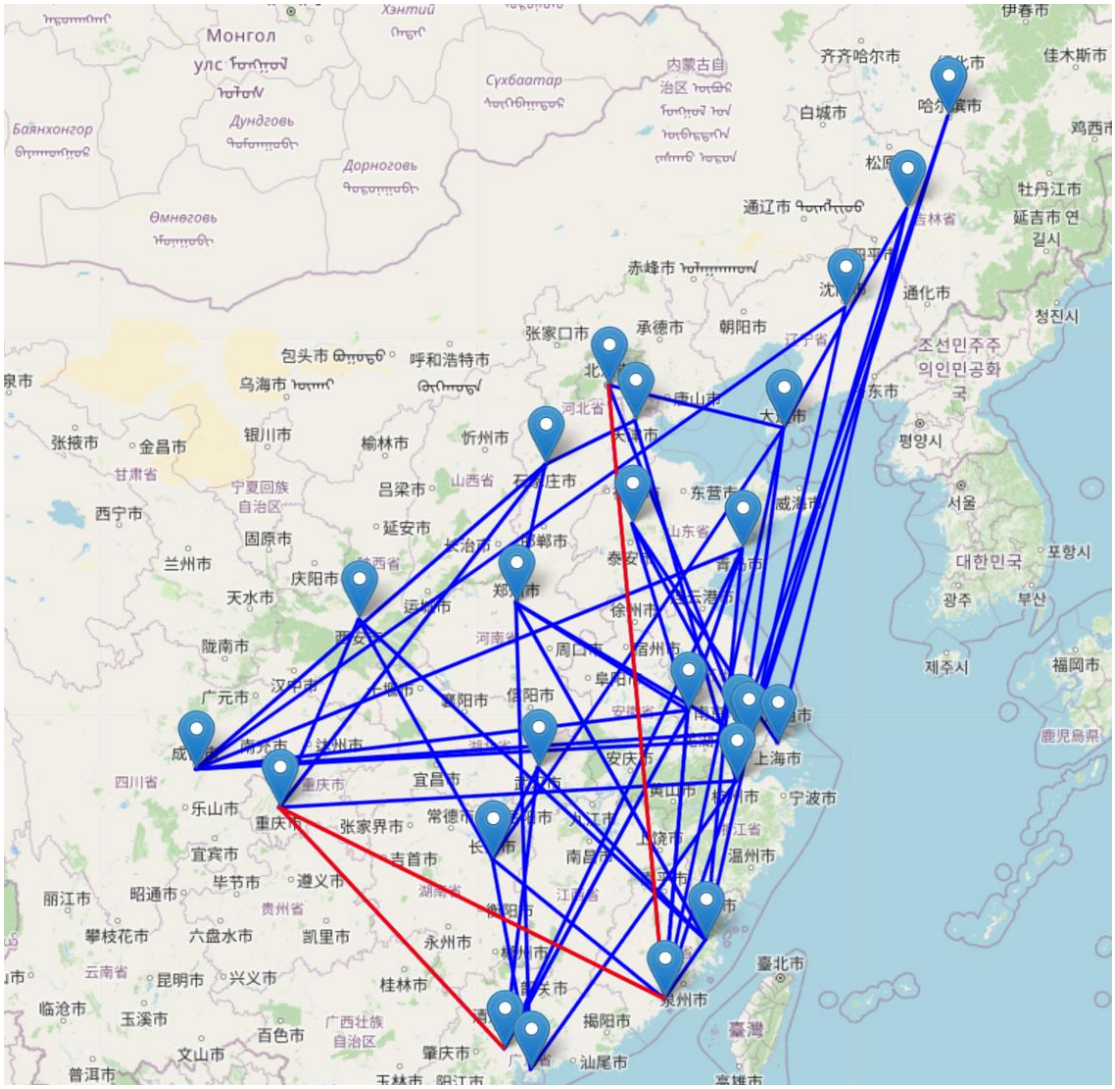


而后需要获取全国各铁路以及桥梁的路径信息，使用爬虫爬取对应信息，代码见附录：

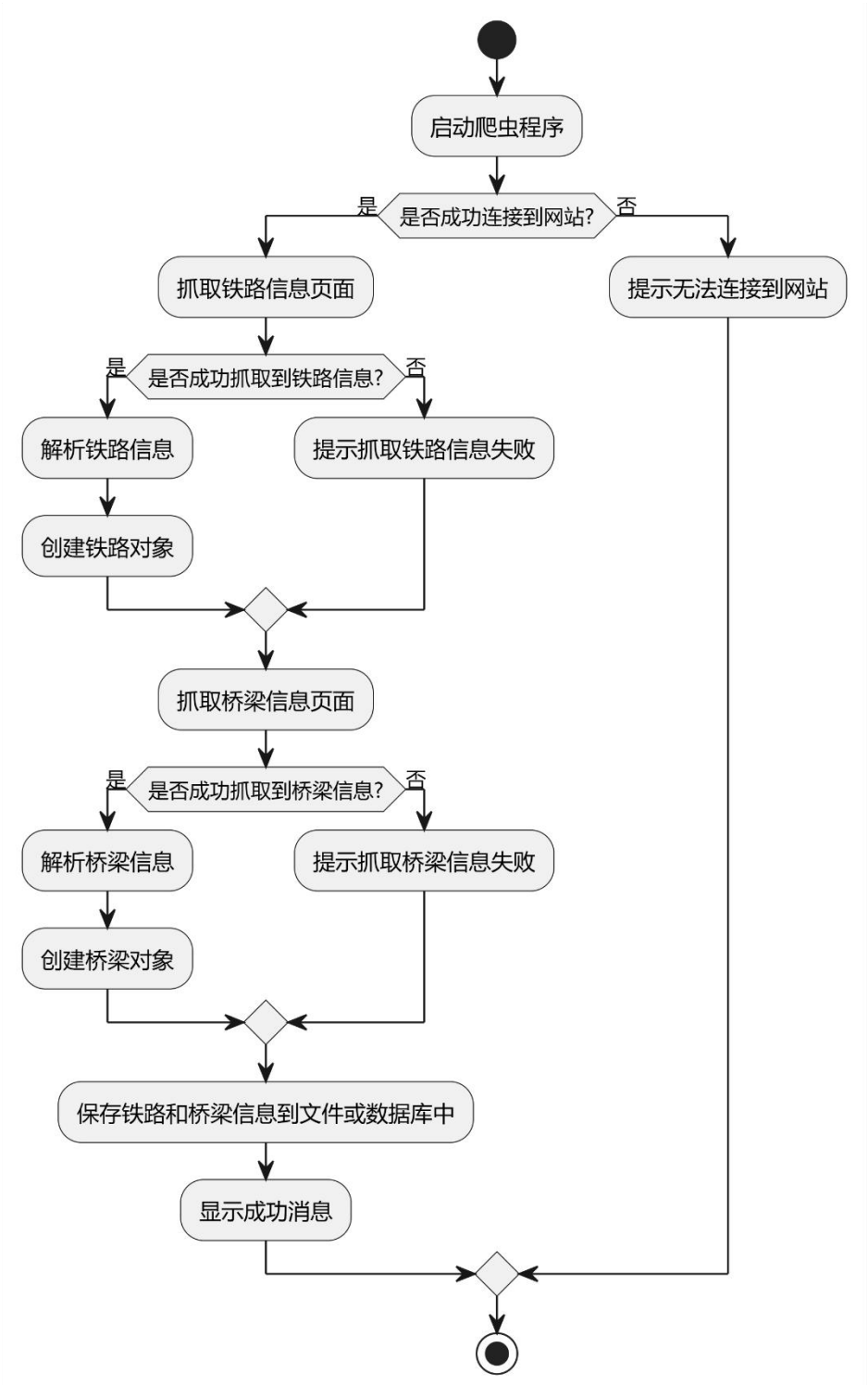


	A	B	C	D
1	温州	S1线	桐岭	
2	大连	1号线	姚家	
3	乌鲁木齐	1号线	国际机场	
4	太原	2号线	尖草坪	
5	兰州	1号线	陈官营	
6	长春	1号线	北环城路	
7	济南	1号线	方特	
8	昆明	1号线(环城	环城南路	
9	东莞	2号线	东莞火车站	
10	南昌	1号线	双港	
11	呼和浩特	1号线	伊利健康谷	
12	厦门	1号线	镇海路	
13	石家庄	1号线	西王	
14	南宁	1号线	火车东站	
15	温州	S1线	潘桥	
16	徐州	1号线	路窝	
17	沈阳	1号线	黎明广场	
18	合肥	1号线	九联圩	
19	郑州	1号线	河南工业大学	
20	无锡	1号线	堰桥	
21	佛山	南海有轨电	礌岗	
22	重庆	1号线	朝天门	
23	哈尔滨	1号线	新疆大街	
24	福州	1号线	象峰	
25	长沙	1号线	开福区政府	
26	南京	S1号线(机	空港新城江宁	
27	北京	S1线	苹果园	
28	大连	1号线	大连北站	
29	深圳	1号线/罗宝	罗湖	
30	杭州	1号线	湘湖	
31	苏州	1号线	木渎	
32	青岛	1号线	东郭庄	
33	宁波	1号线	霞浦	
34	常州	1号线	森林公园	
35	西安	1号线	沣河森林公园	
36	贵阳	1号线	小孟工业园	

最后根据站点和铁路信息设计对应的最短路径算法可视化



六、详细设计（核心代码、算法流程图等）



代码见附录

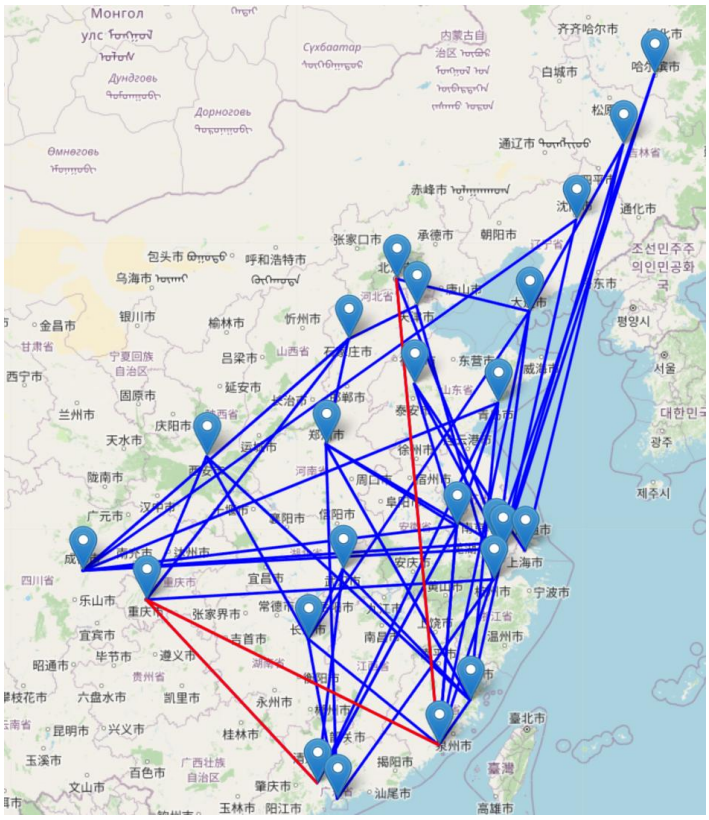
七、测试结果及分析

因爬取的信息有所差异，即铁路和桥梁信息可能有缺，因此所作最短路径分析有所欠缺和不足，仅选取各省会城市作为节点，防止跑崩电脑。

运行程序，先输入需要计算的起点和重点城市名称，而后会自动计算出其间的最短路径和对应的距离，如下所示

```
请输入起点城市：北京市
请输入终点城市：广州市
从 北京市 到 广州市 的最短路径: ['北京市', '厦门市', '重庆市', '广州市']
最短路径距离: 37.42351784294134
```

同时进行可视化分析，对选中的城市的最短路径进行标注，以明确如何规划行程。如下所示。



通过观察可以得知，这条线路确实为最短路线。

八、总结

本次实验成功掌握了爬取网页的技能，同时对调用 api 进行编程也有了一定的认知，设计出的可视化界面也相对较为直观和美观。

但仍有较多不足之处，例如铁路、桥梁信息缺失较多，造成实际中应该存在的部分路线不存在，尽管满足了最短路线设计的要求，但违背了现实生活

同时使用的是最简单的 dijkstra 算法，整个系统的时间成本还是过高，有待后续改善

附录

爬虫：

```
import requests
from bs4 import BeautifulSoup
import csv

# 目标URL（假设这是中国铁路某个页面的URL）
url =
'https://kyfw.12306.cn/otn/leftTicket/init?linktypeid=dc&fs=%E5%8C%97
%E4%BA%AC,BJP&ts=%E8%A5%BF%E5%AE%89,XAY&date=2024-06-10&flag=N,N,Y'

# 发送HTTP GET请求
response = requests.get(url)

# 检查请求是否成功
if response.status_code == 200:
    # 解析HTML内容
    soup = BeautifulSoup(response.text, 'html.parser')

    # 查找包含列车路线的表格或元素（需要根据实际页面结构调整）
    routes = []
    for route in soup.find_all('div', class_='route-class'):  # 假设页
面中有'div'元素包含路线信息
        train_number = route.find('span', class_='train-number').text
```

```
        start_station = route.find('span',
class_='start-station').text
        end_station = route.find('span', class_='end-station').text
        duration = route.find('span', class_='duration').text

        routes.append([train_number, start_station, end_station,
duration])

    # 将数据保存到 CSV 文件
    with open('train_routes.csv', 'w', newline='', encoding='utf-8') as
file:
        writer = csv.writer(file)
        writer.writerow(['Train Number', 'Start Station', 'End Station',
'Duration'])
        writer.writerows(routes)

    print('数据已保存到 train_routes.csv 文件中')
else:
    print('请求失败，状态码：', response.status_code)
```

可视化最短路径：

```
import pandas as pd
import folium
import requests
import random
import math
import heapq

# 百度地图 API 密钥
api_key = 'REDACTED_API_KEY'

# 初始化地图
m = folium.Map(location=[35, 105], zoom_start=5)

# 读取 CSV 文件
csv_file_path = 'subway.csv'
df = pd.read_csv(csv_file_path, encoding='utf-8', errors='ignore')


# 提取所有城市列表
all_cities = set(df.iloc[:, 0]).union(set(df.iloc[:, 2]))
```

```
# 存储地址及其经纬度信息
location_dict = {}

# 获取每个城市的经纬度信息
for city in all_cities:
    # 构造地理编码请求 URL
    geocode_url =
f"http://api.map.baidu.com/geocoding/v3/?address={city}&output=json&a
k={api_key}"

    # 发送请求并获取响应
    response = requests.get(geocode_url)
    data = response.json()

    if data['status'] == 0:
        # 获取经纬度
        lng = data['result']['location']['lng']
        lat = data['result']['location']['lat']
        location_dict[city] = (lat, lng)
        print(f"{city} 经度: {lng}, 纬度: {lat}")

        # 添加标记到地图
        folium.Marker([lat, lng], popup=city).add_to(m)
    else:
        print(f"获取 {city} 的地理编码信息失败")

# 构建图
graph = {city: {} for city in location_dict}
for index, row in df.iterrows():
    start = row.iloc[0]
    end = row.iloc[2]
    start_coords = location_dict[start]
    end_coords = location_dict[end]

    # 计算两点之间的距离
    distance = math.sqrt((start_coords[0] - end_coords[0]) ** 2 +
(start_coords[1] - end_coords[1]) ** 2)

    # 在图中添加边
    graph[start][end] = distance

    # 画线
    folium.PolyLine([start_coords, end_coords], color="blue",
weight=2.5, opacity=1).add_to(m)
```

```
    print(f"路线: {start} -> {end} 距离: {distance}")

# Dijkstra 算法
def dijkstra(graph, start, end):
    # 初始化
    pq = [(0, start)]
    distances = {node: float('inf') for node in graph}
    previous_nodes = {node: None for node in graph}
    distances[start] = 0
    visited = set()

    while pq:
        current_distance, current_node = heapq.heappop(pq)

        if current_node in visited:
            continue

        visited.add(current_node)

        if current_node == end:
            break

        for neighbor, weight in graph[current_node].items():
            distance = current_distance + weight

            if distance < distances[neighbor]:
                distances[neighbor] = distance
                previous_nodes[neighbor] = current_node
                heapq.heappush(pq, (distance, neighbor))

    # 构建最短路径
    path = []
    current_node = end
    while previous_nodes[current_node] is not None:
        path.insert(0, current_node)
        current_node = previous_nodes[current_node]
    if path:
        path.insert(0, current_node)
    return path, distances[end]

# 输入起点和终点城市
start_city = input("北京")
end_city = input("宁波")
```

```
# 计算最短路径
shortest_path, shortest_distance = dijkstra(graph, start_city, end_city)
print(f"从 {start_city} 到 {end_city} 的最短路径: {shortest_path}")
print(f"最短路径距离: {shortest_distance}")

# 在地图上绘制最短路径
path_coords = [location_dict[city] for city in shortest_path]
folium.PolyLine(path_coords, color="red", weight=2.5,
opacity=1).add_to(m)

# 保存地图到 HTML 文件
m.save('baidu_map_routes.html')

# 输出地图
m
```